

CS102: Data Structures and Algorithms

Week 1: Foundations of Data Structures

Milestone 1: Foundations of Structure & Index Mechanics

In computer science, a data structure is a specialized format engineered for organizing, storing, and retrieving data efficiently. Selecting an optimal structure depends entirely on the layout patterns and retrieval methods demanded by your application architecture.

Lessons 2 & 4: Structural Selection — Arrays vs. Dictionaries

Data structures bifurcate based on access mechanics: position-based sequential tracks (arrays or Python lists) and custom label-based mapping systems (dictionaries).

■ Expanded Content: Structural Allocation Rules

- **Arrays / Lists:** An ordered row of sequential slots. Each slot is accessible solely by its fixed positional index, making it ideal for groups of homogeneous values.
- **Dictionaries:** Labeled address spaces mapping unique, custom keys to distinct values, optimized for composite, multi-attribute profiles.

```
# Allocation Choice: Position-Based Array vs. Custom Labeled Mapping
temperature_array = [7, 18, 75, 35] # Positional chronological order

student_dictionary = {
    "name": "Jordan",
    "age": 21,
    "email": "jordan@example.com"
}
```

Lessons 3 & 5: Positional Access & Complexity Baselines

Retrieving values from an index-based data structure occurs via zero-based indexing or sequential linear scanning. These methods carry starkly different performance costs as memory structures expand.

■ Expanded Content: Time Complexity Mapping

- **Direct Index Access — $O(1)$:** Jumping straight to a known address slot (e.g., `items[3]`). The operations cost remains constant, completely independent of total array size.
- **Sequential Scan — $O(n)$:** Traversing the array slot-by-slot from index 0 when the target index is unknown. The operational cost scales linearly with total element volume.

```
# O(1) Constant Lookup vs. O(n) Linear Search Scan Execution
numbers_list = list(range(1, 100001))

# Direct Index Access: Constant Cost O(1)
immediate_value = numbers_list[50000]

# Sequential Scan Search: Linear Cost O(n)
target_value = 50001
for element in numbers_list:
    if element == target_value:
        break
```

Milestone 2: Element Manipulation & Operational Sequencing

Modifying an index-based array requires combining values, inserting items, and handling deletions while precisely tracking memory index shifts.

Lessons 6 & 7: Mutation Mechanics — Updates, Slippage, and Aliases

Updating values by index modifies memory inline without duplicating the underlying object array. However, assigning multiple variables to the same structure introduces aliasing traps where mutations scale down shared references. Removing data causes immediate structural index slippage.

```
# The Aliasing Trap and Sequential Value Deletion Shifts
original_scores = [10, 20, 30, 40]
aliased_reference = original_scores

# Updates change both references simultaneously
aliased_reference[0] = 99 # original_scores[0] is now 99

# Index Slippage: Popping index 1 drops 20; shifts 30 down to index 1
popped_value = original_scores.pop(1) # Returns 20
```

Lessons 9 & 10: Multi-Step Operational Pipelines

Combining accesses, updates, and removals in a single execution loop requires grouping structural changes within repeatable, modular functions.

```
# Pipeline Sequence: Access, Update, and Tail-Removal
def transform_pipeline(dataset):
    initial_capture = dataset[0]
    dataset[1] = "processed_node"
    removed_element = dataset.pop() # Drop tail block
    return initial_capture, removed_element, dataset

capture, dropped, modified_data = transform_pipeline(["start", "temp", "end"])
```

Milestone 3: High-Dimensional Topologies & Boundary Engineering

Complex physical grids, coordinate layers, and spatial matrices require nesting multi-level list shapes protected by reliable defensive barriers.

Lessons 11 & 12: High-Dimensional Array Implementations

High-dimensional arrays are multi-level lists where navigating down to a raw element requires chaining multiple independent index selectors.

Topology Shape	Chained Selector Mapping	Optimal System Application Example
1-D Line Track	array[position]	Flat chronological event streams, message logs, and score lists.
2-D Spatial Grid	grid[row][column]	Seating layouts, coordinate game matrices, and image pixel buffers.
3-D Volumetric Cube	cube[layer][row][column]	Volumetric tracking coordinates, game map layers, and multi-level warehouses.

```
# Chained Coordinate Updates in a 3-D Warehouse List
warehouse_matrix = [
    ["item_00", "empty"], ["empty", "empty"]], # Level 0 Grid
    ["empty", "empty"], ["empty", "empty"] # Level 1 Grid
]

# Update coordinate: Level 0, Row 0, Column 1
warehouse_matrix[0][0][1] = "payload"
```

Lesson 13: Edge Case Isolation & Defensive Guards

Attempting to read or mutate array spots that sit completely outside bounded address slots triggers immediate runtime crashes (IndexError). Defensive code requires strict check filters before reading data.

```
# Bounded Scoreboard Guard Array Generator
def generate_safe_top_three(unfiltered_names):
    calibrated_scoreboard = []
    for rank_index in range(3):
        if rank_index < len(unfiltered_names):
            calibrated_scoreboard.append(unfiltered_names[rank_index])
        else:
            calibrated_scoreboard.append("---")
    return calibrated_scoreboard

empty_run = generate_safe_top_three([]) # Output: ["---", "---", "---"]
```